

Chapter 4

CSS Layout with Flexbox and Grid Part 2

INT150 Web Technologies (2/2025)

Sanit Sirisawatvatana

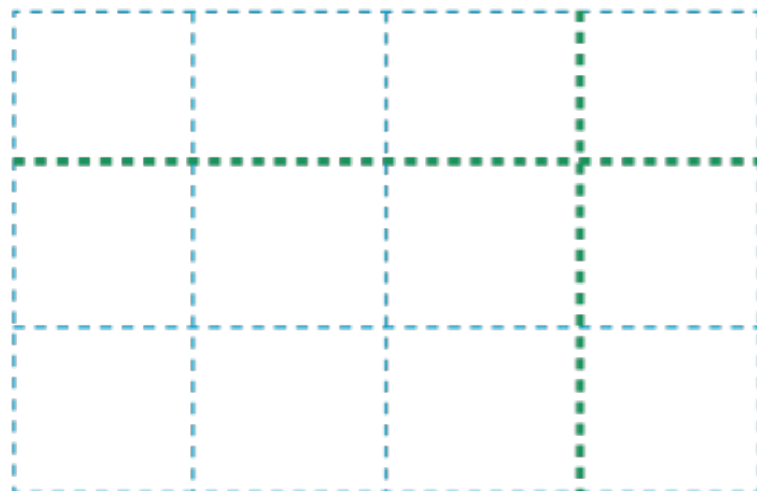
CSS Layout with Grid

- The CSS Grid Layout Module
 - provides a system for laying out elements in rows and columns (while flexbox lays out elements on one axis only)
 - can remain completely flexible to fit a variety of screen sizes or mimic a print page layout.

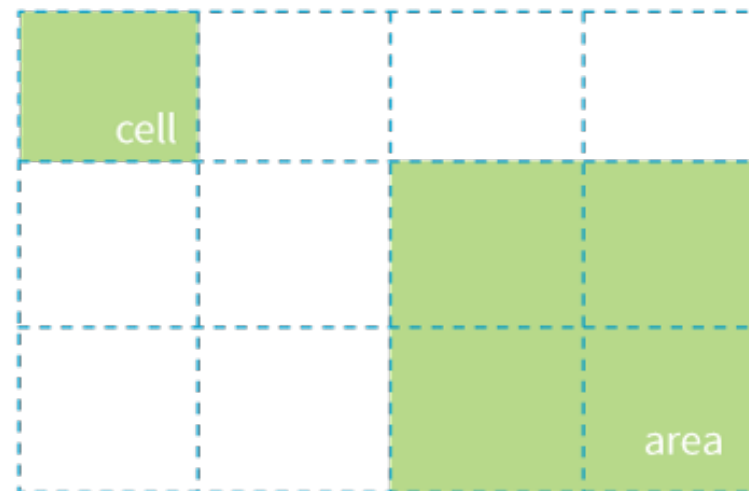
Steps for using Grid

1. Set an element's display to grid to establish a grid container. Its children become grid items.
2. Set up the columns and rows for the grid (explicitly or with rules for how they are created on the fly).
3. Assign each grid item to an area on the grid (or let them flow in automatically in sequential order).

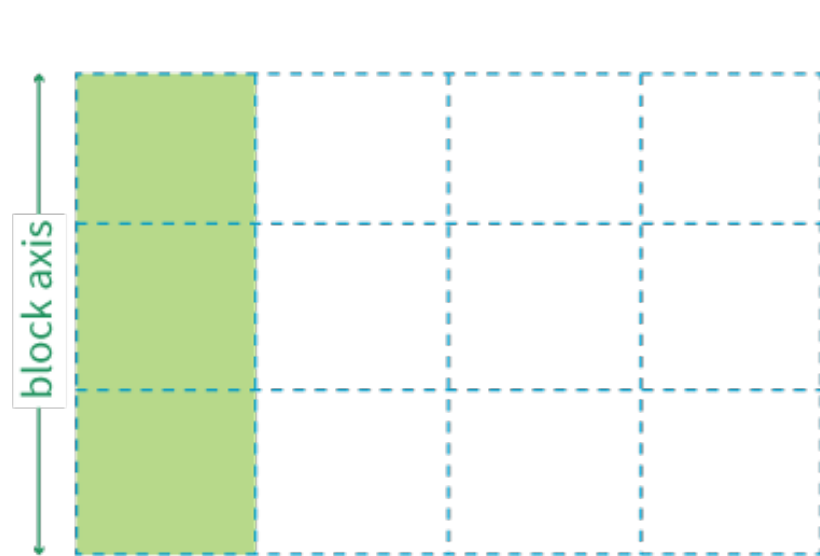
Grid Terminology



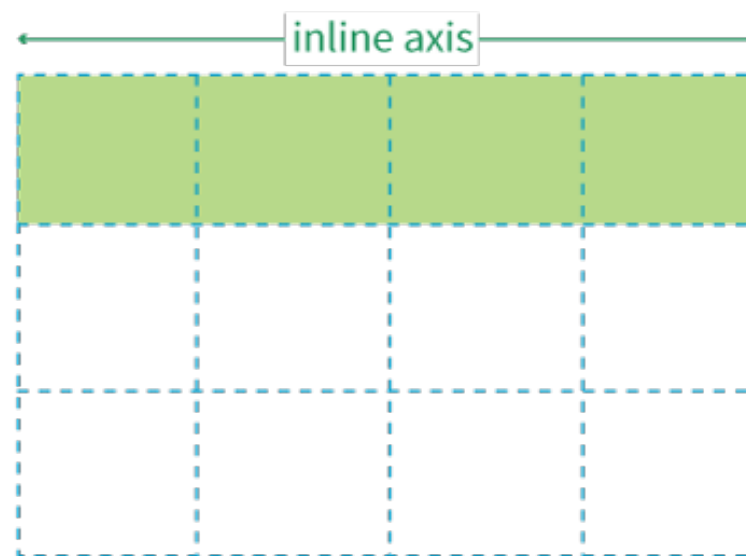
grid lines



grid cell and grid area



grid track (column)



grid track (row)

Grid Terminology

- Grid line
 - The horizontal and vertical dividing lines of the grid
- Grid cell
 - The smallest unit of a grid is a grid cell, which is bordered by four adjacent grid lines with no grid lines running through it.
- Grid area
 - A grid area is a rectangular area made up of one or more adjacent grid cells.
- Grid track
 - The space between two adjacent grid lines is a grid track, which is a generic name for a [grid column](#) or a [grid row](#).
- Block axis
 - [Grid columns](#) go along the block axis, which is vertical (as block elements are stacked) for languages written horizontally.
- Inline axis
 - [Grid rows](#) follow the inline (horizontal) axis.

Declaring Grid Display

- To turn an element into a grid container

`display : grid (or inline-grid)`

- All of its children automatically become grid items.

The markup

```
<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
</div>
```

The styles

```
#layout {
  display: grid;
}
```

Setting up the columns and rows for the Grid

grid-template-rows

grid-template-columns

Values:	none list of track sizes and optional line names
---------	--

Default:	none
----------	------

Applies to:	grid container
-------------	----------------

Inherits:	no
-----------	----

- The value of `grid-template-rows` is a list of the `heights` of each row track in the grid.
- The value of `grid-template-columns` is a list of `widths` of each column track in the grid.
- The `number of track sizes` determines `the number of rows` or `columns`.

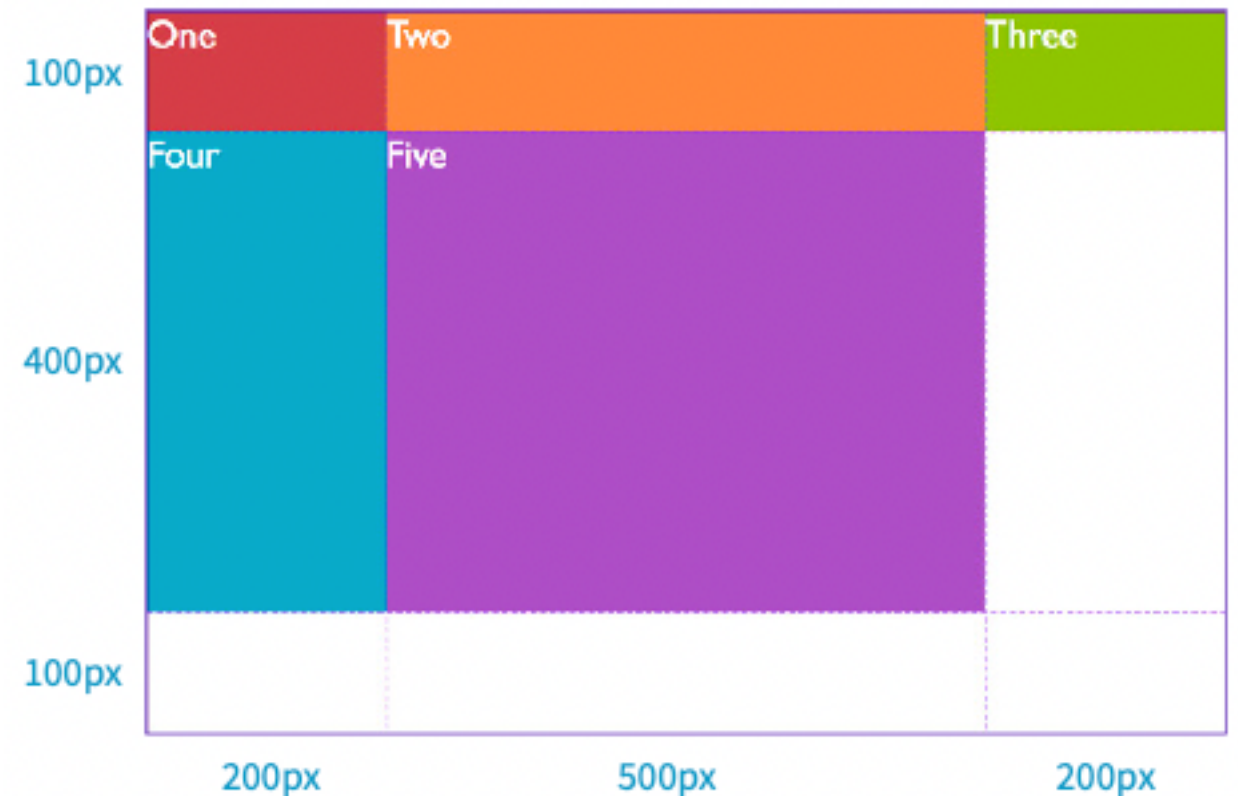
Grid Container

The markup

```
<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
</div>
```

The styles

```
#layout {
  display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px 500px 200px;
}
```

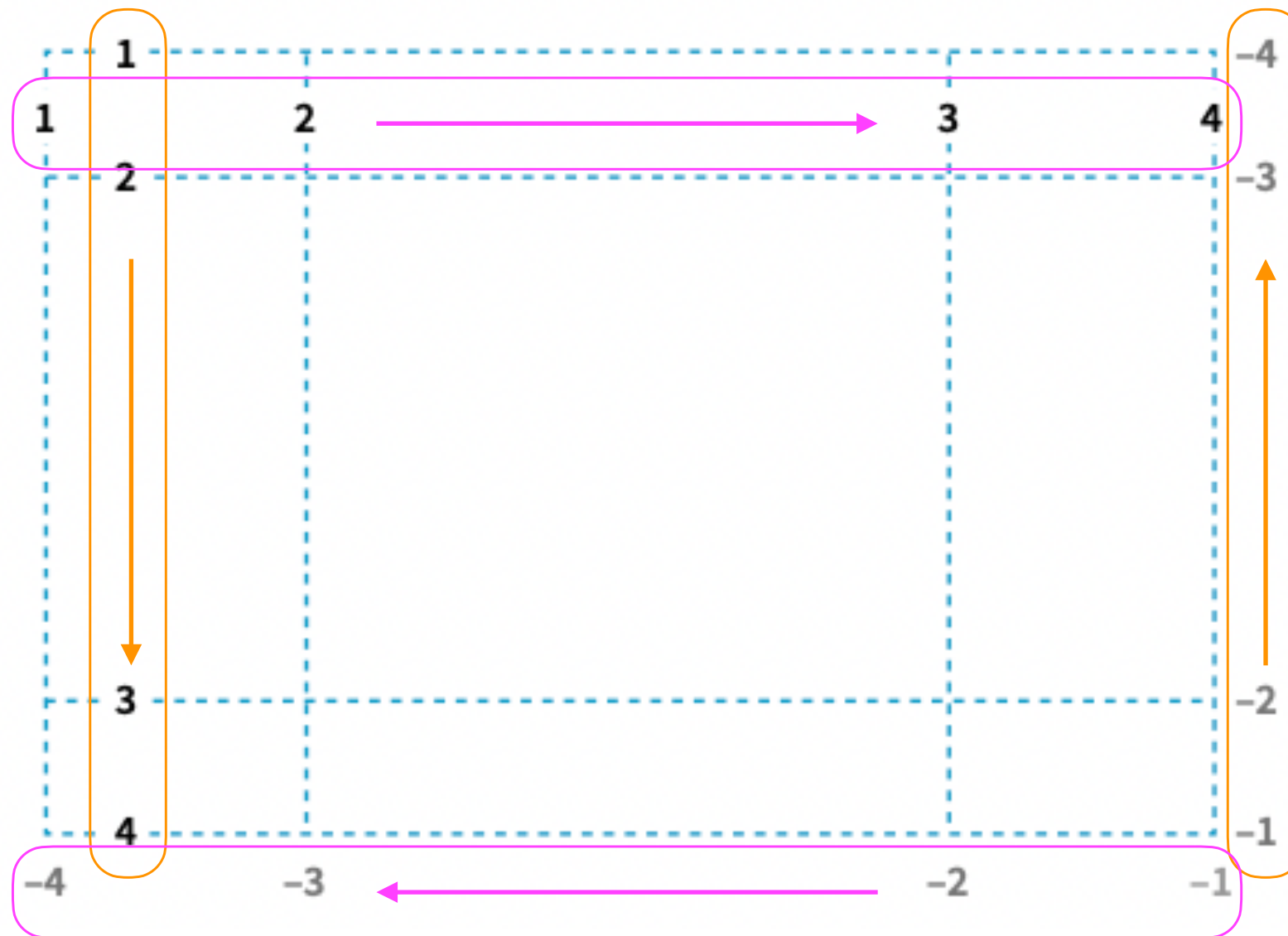


- The template properties divide the `#layout` container into **three columns** and **three rows**.
- By default, the **grid items** flow in order into the available grid cells.
- Because there are only five child elements in the `#layout` div, only the first five cells are filled.

Grid line numbers and names

- When the browser creates a grid, it automatically assign each grid line a number (called **a grid line number**) that you can reference when positioning items.
- The grid line at the **start** of the grid track is **1** and lines are numbered sequentially from there.
- The lines are numbered **from the end of tracks** as well, starting with **-1**, and numbers count back from there (-2, -3, etc).
- Being able to target the end of a row or column without counting lines is very useful feature.

Grid line numbers



Grid Line Names

- You can also assign names to lines to make them more intuitive to reference later.
- Grid line names are added in square brackets in the position they appear relative to the tracks.

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [content-start] 400px  
  [footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 500px [links] 200px;  
}
```

- The grid line at the top of the grid can now be referenced to as "**header-start**", "**1**" or "**-4**".

Grid Line Names

- To give a line more than one name, include all the names in brackets, separated by spaces

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [header-end content-start]  
400px [content-end footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 500px [links] 200px;  
}
```

- It is common for each grid line to end up with multiple names and numbers and you can choose whichever is easiest to use.
- We'll be using these numbers and names to place items on the grid.

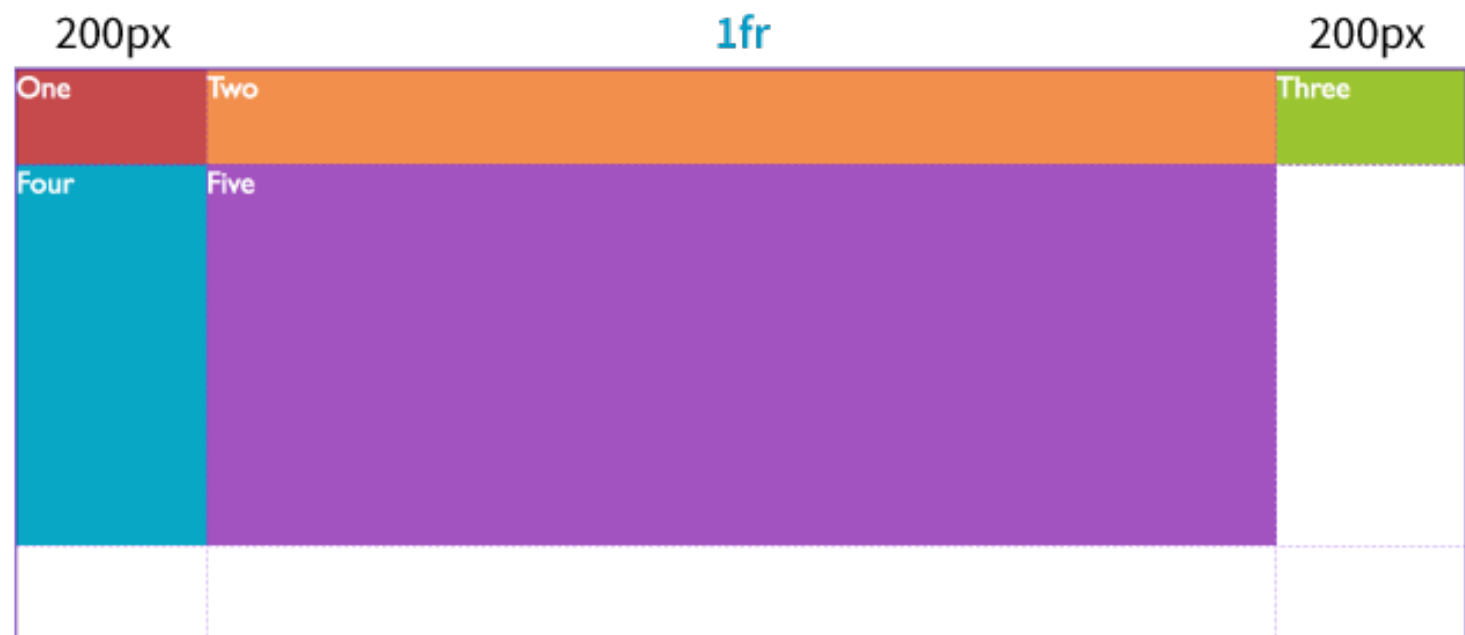
Track Size values

- The CSS Grid spec provides a lot of ways to specify the [width](#) and [height](#) of a track. Some of these ways to adapt to available space and/or to the content they contain:
 - Lengths (such as px or em)
 - Percentage values (%)
 - [Fractional unit \(fr\)](#)
 - [minmax\(\)](#)
 - [min-content, max-content](#)
 - [auto](#)
 - [fit-content\(\)](#)
- The [repeat\(\)](#) function that allows you to set up a repeating pattern of track widths.

Fractional Units (fr)

- The Grid-specific **fractional unit (fr)** allows developers to create track widths that **expand** and **contract** depending on **available space**.

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 1fr 200px;  
}
```



Fractional Units (fr)

- The **fr unit** is great for combining **fixed** and **flexible track widths**, but you also could use **all fr units** to give **all the columns proportional widths**.

```
grid-template-columns: 1fr 2fr 1fr;
```

- All of the column widths flex according to the available browser width, but the middle column will always be twice the width of the side columns.

Size Range with minmax()

- The `minmax()` function constricts the size range for the track by setting a minimum and maximum dimension.
- It is used in place of a specific track size.
- This rule sets the middle column to at least 15em but never more than 45em.

```
grid-template-columns: 200px minmax(15em, 45em) 200px;
```


Content-based sizing

- The `min-content`, `max-content` and `auto` values size the track based on the size of content within it.
- `min-content` is the smallest that a track can get without overflowing (`the width of the longest word or widest image`).
- `max-content` is the maximum amount of space needed for the content. When used as a column width, the column track will be as wide as the widest content in that track without line wrapping.
- `auto` lets the browser take care of it. Start with `auto` for content-based sizing.

min-content and max-content

Text content in cell

Look for the good in
others and they'll see
the good in you.

Column width set to
`max-content`

Look for the good in others and they'll see the good in you.

Column width set to
`min-content`

Look
for
the
good
in
others
and
they'll
see
the
good
in
you.

Repeating Track Sizes

- The shortcut `repeat()` function lets you repeat patterns in track sizes:

`repeat( #, track pattern)`

- The first number is the number of repetitions. The track sizes after the comma provide the pattern:

BEFORE:

```
grid-template-columns: 200px 20px 1fr 20px 1fr 20px 1fr 20px 1fr 20px  
1fr 200px;
```

AFTER:

```
grid-template-columns: 200px repeat(5, 20px 1fr) 200px;
```

- You can also provide grid line names before and/or after each track size, and those names will also be repeated in the pattern:

```
grid-template-rows: repeat(4, [date] 5em [event] 1fr);
```

Repeating Track Sizes

- You can let the browser figure out how many times a repeated pattern will fit with `auto-fill` and `auto-fit` values instead of a number:
- `auto-fill` creates as many tracks as will fit in the available space, even if there's not enough content to fill all tracks.
- `auto-fit` creates as many tracks as will fit, dropping empty tracks from the layout.

auto-fill and auto-fit

For example: the grid container is 35 tall

```
grid-template-rows: repeat(auto-fill, 10em);
```

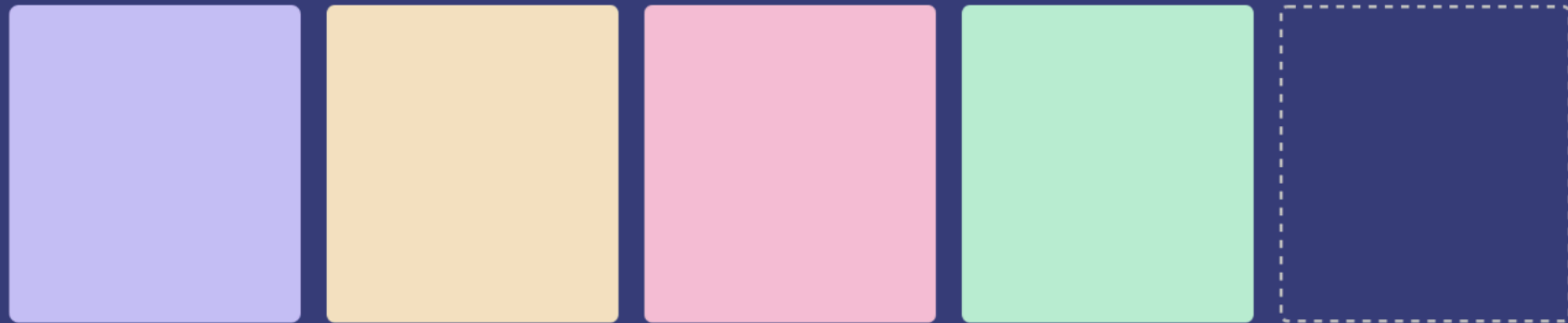
- The browser creates a row every 10ems until it runs out of room, resulting in three rows. Even if there is only enough content to fill the first rows, all three rows are created and the space is held in the layout.
- The `auto-fit` value works similarly, except any tracks that do not have content get dropped from the layout. If there is leftover space, it is distributed according to the vertical (`align-content`) and horizontal (`justify-content`) alignment values provided.

auto-fill vs. auto-fit

auto-fill

```
repeat(auto-fill, minmax(200px, 1fr));
```

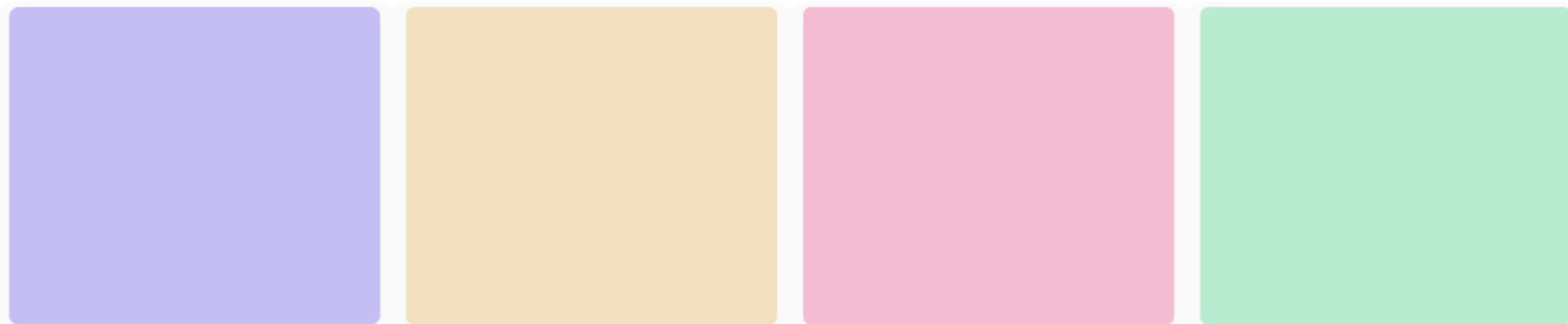
Don't auto expand the cols when more space is available



auto-fit

```
repeat(auto-fit, minmax(200px, 1fr));
```

Expand the cols when more space is available



Defining grid area

grid-template-areas

Values: none | series of area names by rows

Default: none

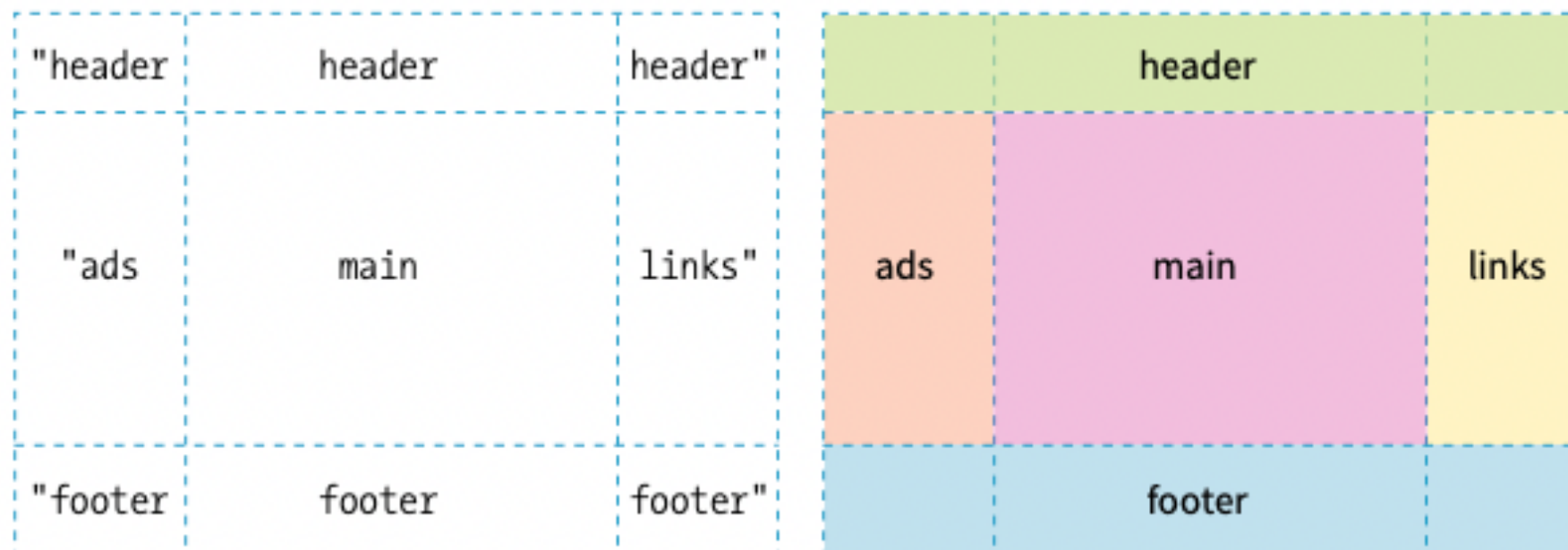
Applies to: grid containers

Inherits: no

- You can also assign names to [areas](#) of the grid.
- A [grid area](#) is made up of [one or more cells](#) in [a rectangle](#) (no L-shapes or other non-rectangular collections of cells).
- The value is a [list of names for every cell](#) in the grid, [listed by row](#).
- When neighboring cells share a name, they form a grid area with that name.

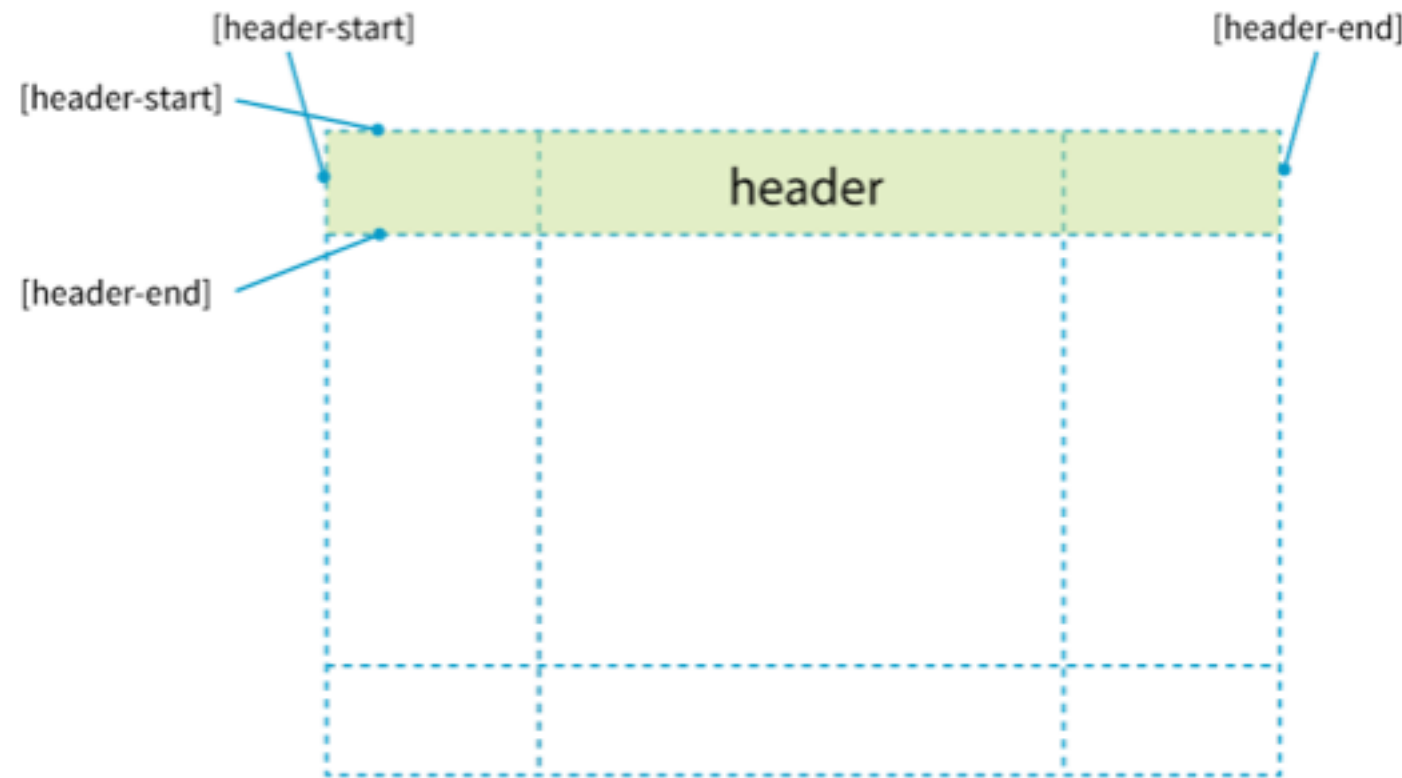
Giving Names to Grid Areas

```
#layout {  
  display: grid;  
  grid-template-rows: [header-start] 100px [content-start] 400px  
[footer-start] 100px;  
  grid-template-columns: [ads] 200px [main] 1fr [links] 200px;  
  grid-template-areas:  
    "header header header"  
    "ads      main  links"  
    "footer footer footer"  
}
```



Giving Names to Grid Areas

- Assigning names to lines with -start and -end suffixes creates an area name implicitly.



- Similarly, when you specify an area name with grid-template-areas, line names with -start and -end suffixes are implicitly generated.

The grid property (shortcut)

grid

Values: none | row info / column info

Default: none

Applies to: grid containers

Inherits: no

- The [grid](#) shortcut sets values for [grid-template-rows](#), [grid-template-columns](#), and [grid-template-areas](#).
- Any properties that are not specified will be reset to their default.
- In grid, the row values and column values are separated by a slash, with the row values appearing first.

The grid property (shortcut)

- Example:

```
#layout {  
  display: grid;  
  grid: 100px 400px 100px / 200px 1fr 200px;  
}
```

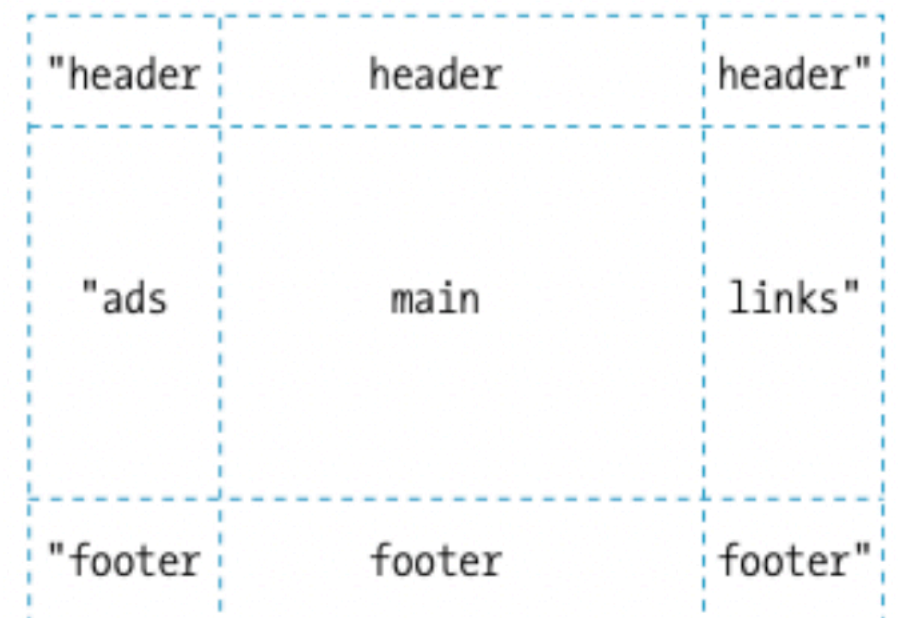
- You can include line names and area names as well, in this order:

[start line name] "area names" <track size> [end line name]

- The line names and area names are optional.

- Example:

```
#layout {  
  display: grid;  
  grid:  
    [header-start] "header header header" 100px  
    [content-start] "ads main links" 400px  
    [footer-start] "footer footer footer" 100px  
  / [ads] 200px [main] 1fr [links] 200px;  
}
```



- The names and height for each row are stacked here for clarify. Note that the column track information is still after the slash.

Placing Items Using Grid Lines

grid-row-start
grid-row-end
grid-column-start
grid-column-end

Values: `auto` | grid line | span number | span "line name" | number "line name"

Default: `auto`

Applies to: grid items

Inherits: `no`

- These properties position grid items on the grid by referencing the grid lines where they begin and end.

Placing Items on the Grid

By line number:

```
#one {  
  grid-row-start: 1;  
  grid-row-end: 2;  
  grid-column-start: 1;  
  grid-column-end: 4;  
}
```

Using a span:

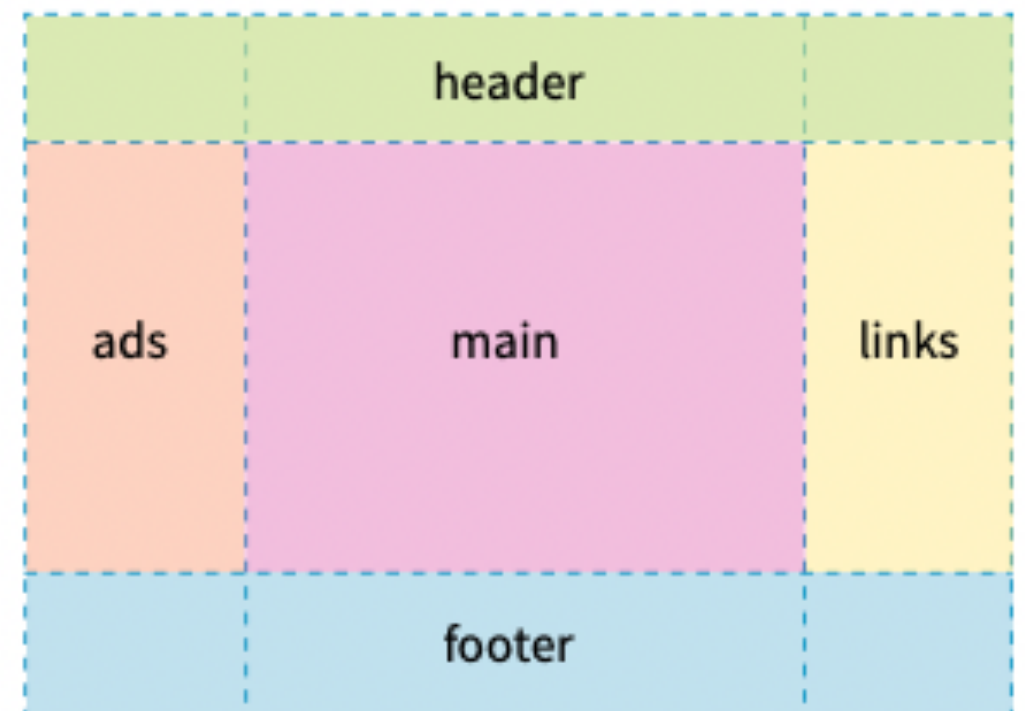
```
#one {  
  ...  
  grid-column-start: 1;  
  grid-column-end: span 3;  
}
```

Starting from the last grid line and spanning backward:

```
#one {  
  ...  
  grid-column-start: span 3;  
  grid-column-end: -1;  
}
```

By line name:

```
#one {  
  grid-row-start: header-start;  
  grid-row-end: header-end;  
  ...  
}
```



grid-row and grid-column (shortcut)

grid-row
grid-column

Values: start line / end line

Default: see individual properties

Applies to: grid items

Inherits: no

- The `grid-row` and `grid-column` properties combine the *-start and *-end properties into a single declaration.
- The start and end line values are separated by a slash (/).

```
#one {  
  grid-row: 1 / 2;  
  grid-column: 1 / span 3;  
}
```

Placing Items on the Grid Using Areas

grid-area

Values: area name | 1 to 4 line identifiers

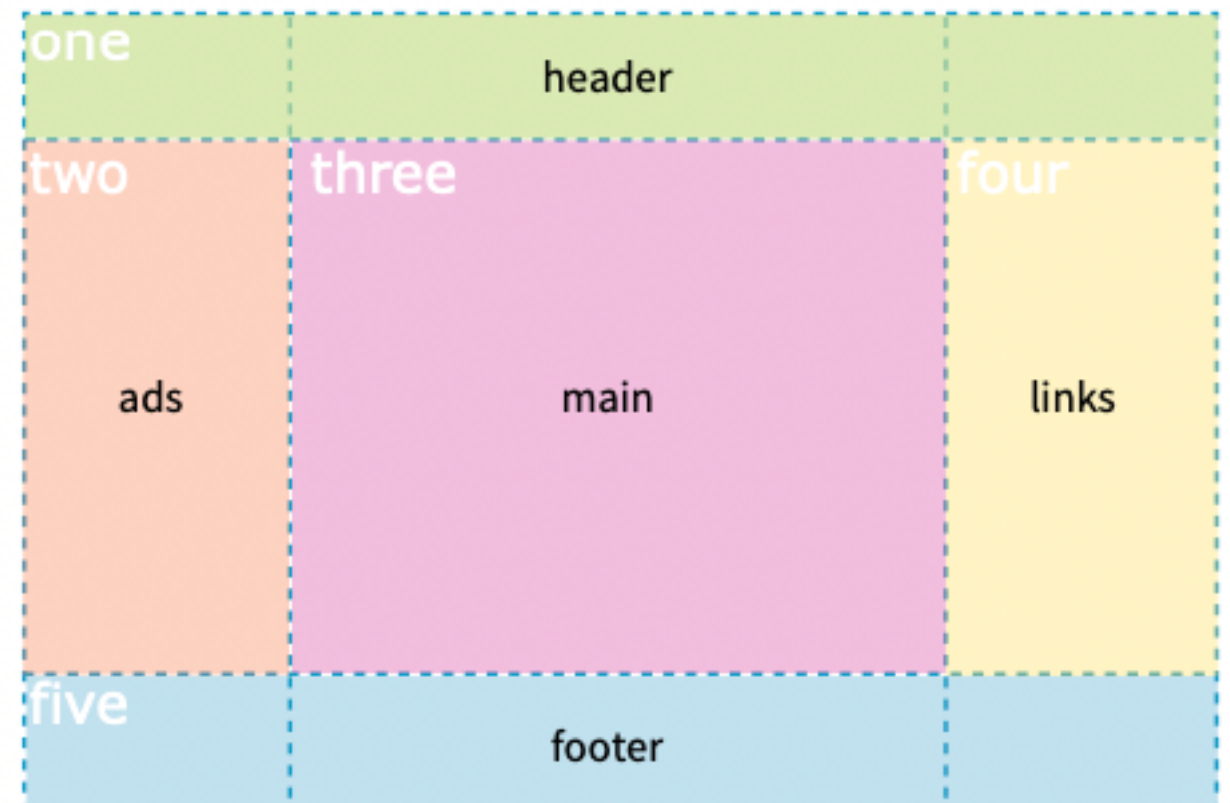
Default: see individual properties

Applies to: grid items

Inherits: no

```
#one { grid-area: header; }
#two { grid-area: ads; }
#three { grid-area: main; }
#four { grid-area: links; }
#five { grid-area: footer; }

#one {
  grid-area: 1 / 1 / 2 / span 3 ;
  /* row-start/column-start/row-end/column-end
  (counterclockwise around the area)
}
```



<https://developer.mozilla.org/en-US/docs/Web/CSS/grid-area>

Implicit Grid Behavior

- The Grid Layout system does some things for you automatically ([implicit behavior](#)):
 - Generating "-[start](#)" and "-[end](#)" line names when you name an area (and vice versa).
 - [Flowing](#) item into grid cells [sequentially](#) if you don't explicitly place them.
 - [Adding rows and columns on the fly](#) as needed to fit items that don't fit the defined grid. For example:
 - If you place an item outside a defined grid, the browser automatically generates tracks in the grid to accommodate it.
 - If you have more items than there are cells or areas, the browser generates more tracks until all the items are placed.

Automatically Generated Tracks

- **By default**, any row or column automatically added to a grid will have the size **auto**, sized just large enough to accommodate the **height** or **width** of **the contents**.
- If you want to give implicit rows and columns specific dimensions, use the **grid-auto-rows** or **grid-auto-columns**.

Automatically Generated Tracks

grid-auto rows , grid-auto-columns

Values: list of track sizes

Default: auto

Applies to: grid containers

Inherits: no

- Provide one more track sizes for automated tracks. If you provide **more than one value**, it acts as **a repeating pattern**.
- **Example** : Column widths are set explicitly with a template, but columns will be generated automatically with a height of 200 pixels:

```
grid-template-columns: repeat(3, 1fr);  
grid-auto-rows: 200px;
```

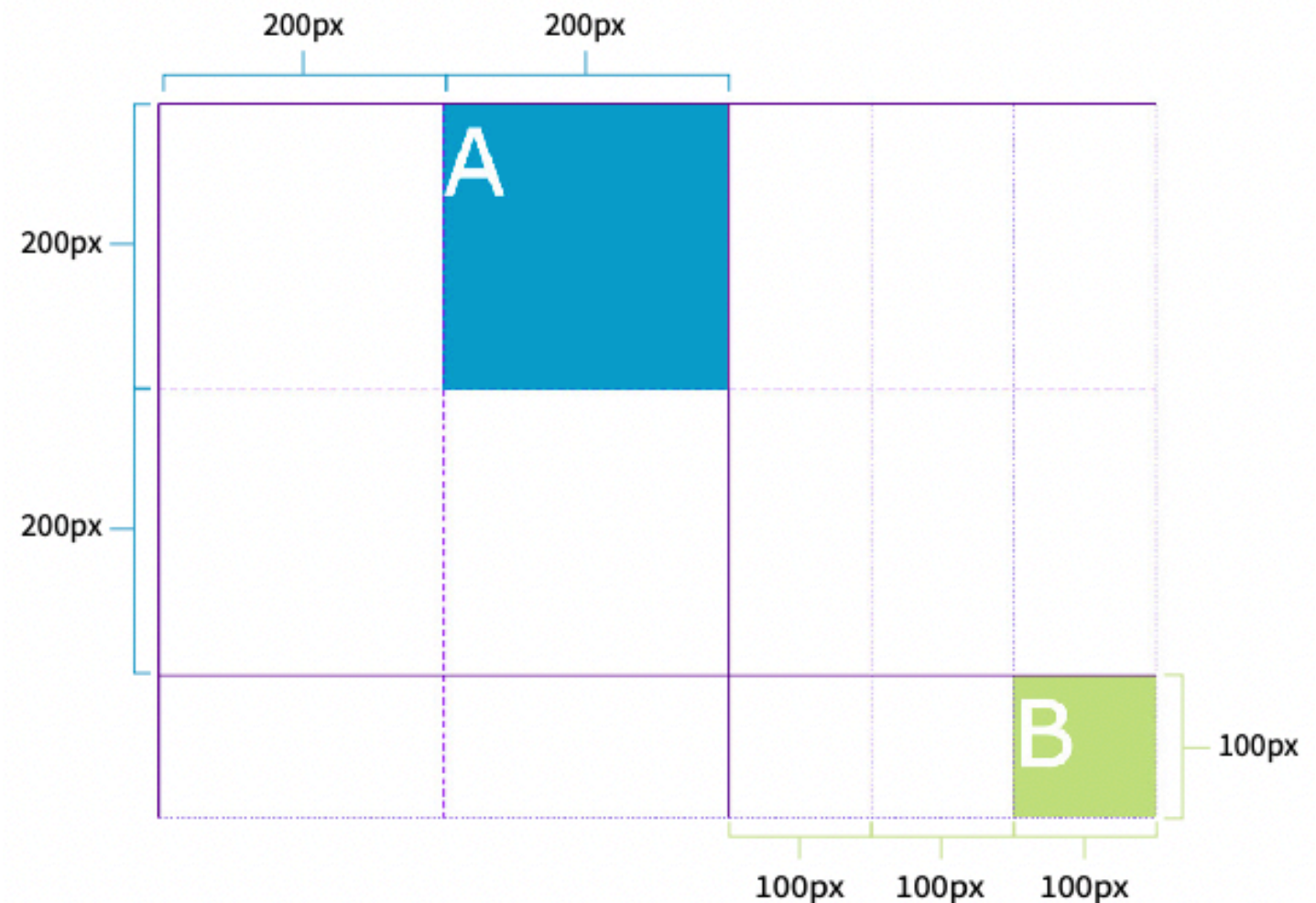
Automatically Generated Tracks

THE MARKUP

```
<div id="littlegrid">
  <div id="A">A</div>
  <div id="B">B</div>
</div>
```

THE STYLES

```
#littlegrid {
  display: grid;
  grid-template-columns: 200px 200px;
  grid-template-rows: 200px 200px;
  grid-auto-columns: 100px;
  grid-auto-rows: 100px;
}
#A {
  grid-row: 1 / 2;
  grid-column: 2 / 3;
}
#B {
  grid-row: 3 / 4;
  grid-column: 5 / 6;
}
```



- Row and column tracks are added automatically as needed. They are sized as specified by `grid-auto-rows` and `grid-auto-columns` (100 pixels)

Flow direction and density

grid-auto-flow

Values: **row** or column | dense (optional)

Default: row

Applies to: grid containers

Inherits: no

- Specifies whether you would like items to flow in by **row** or **column**. The **default** is the **writing direction** of the document.

- Example:

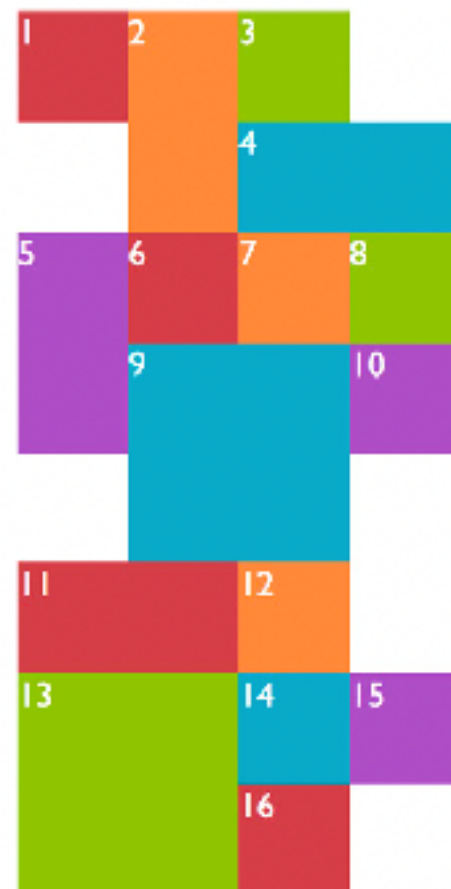
```
#listings {  
  display: grid;  
  grid-auto-flow: column ;  
}
```

Flow direction and density

- **By default**, items are placed in the first area in which they fit.
- Cells that are too small to accommodate the content will be skipped over until a cell large enough is found for placement.
- If you include the optional **dense** keyword for the **grid-auto-flow** property, it instructs the browser to fill the grid as densely as possible.
 - This allows the items to appear out of sequence in order to fill the available space.

Flow direction and density

Default flow pattern



Dense flow pattern



```
#listings {  
  display: grid;  
  grid-auto-flow: row dense ;  
}
```

The grid property

- Use the **auto-flow** keyword in the shortcut **grid** property to indicate that the **rows** and **columns** should be **generated automatically**.
- Example: Columns are established explicitly, but the row generate automatically.

```
grid: auto-flow 200px / repeat(3, 1fr);
```

- Because **auto-flow** is included with row information, **grid-auto-flow** is set to **row**.

```
grid: 300px 300px / auto-flow 100px;
```

- The grid will have two 330px rows, but 100px-wide columns will be generated on the fly as grid items are added. The **grid-auto-flow** is set to **column**.

Spacing and Alignment

- You can add space between tracks and adjust alignment of the grid and its items by using many of the same methods for Flexbox.

`grid-row-gap` , `grid-column-gap`

Values:	length (must not be negative)
---------	-------------------------------

Default:	0
----------	---

Applies to:	grid containers
-------------	-----------------

Inherits:	no
-----------	----

`grid-gap`

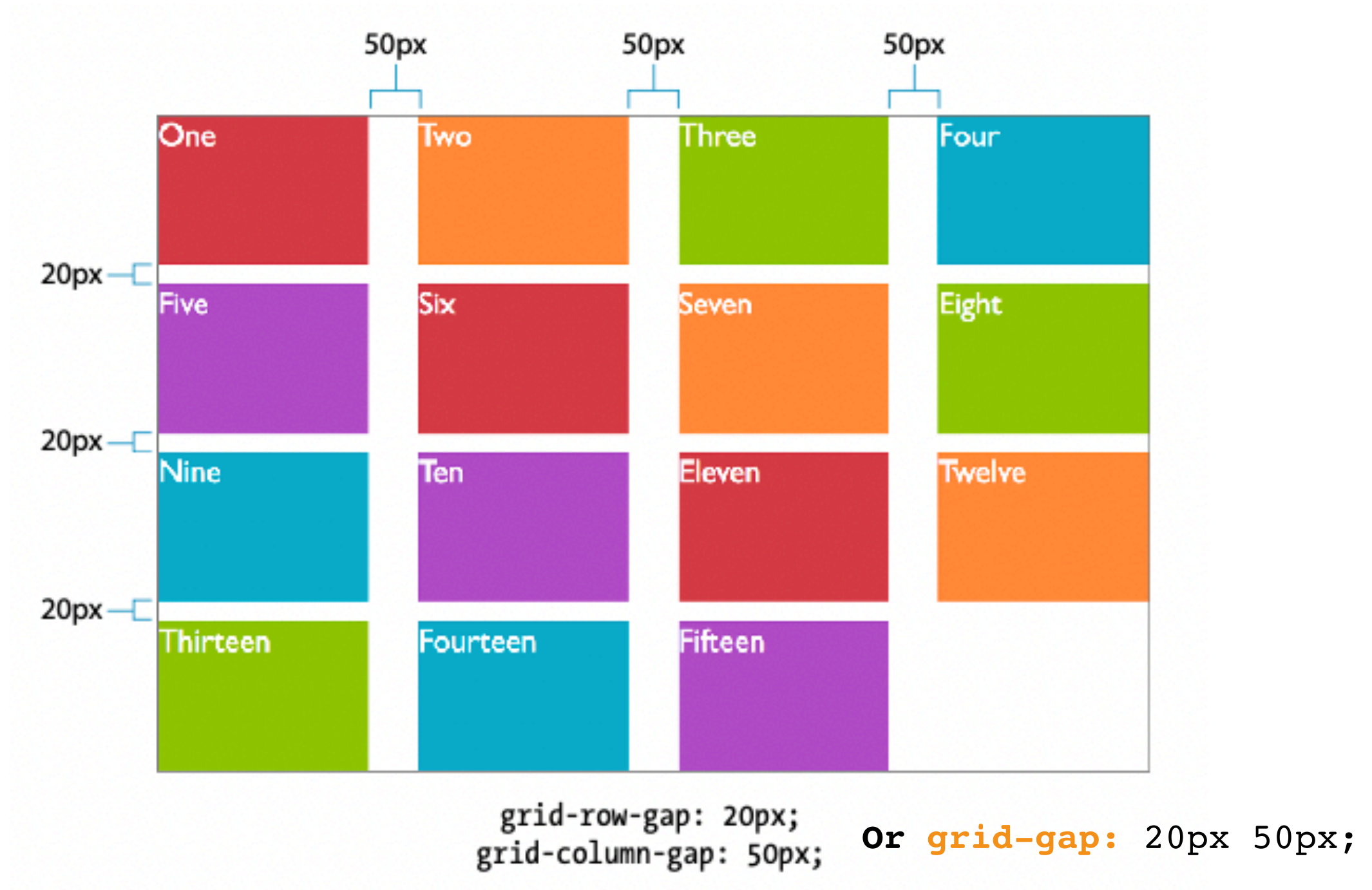
Values:	<code>grid-row-gap</code> <code>grid-column-gap</code>
---------	--

Default:	0 0
----------	-----

Applies to:	grid containers
-------------	-----------------

Inherits:	no
-----------	----

Grid gaps add gutter spaces between tracks



Align individual items

- When a grid item doesn't fill its entire grid area, you can specify how you'd like it to be aligned in that space.
- Specify the **horizontal (inline) alignment** with the **justify-self** property.
- **align-self** specifies **alignment** on the **vertical (block) axis**.

Aligning individual items

justify-self

Values: start | end | center | left | right | self-start | self-end | stretch | normal | [auto](#)

Default: auto (looks at the value for justify-items, which defaults to normal)

Applies to: grid items

Inherits: no

align-self

Values: start | end | center | left | right | self-start | self-end | stretch | normal | auto

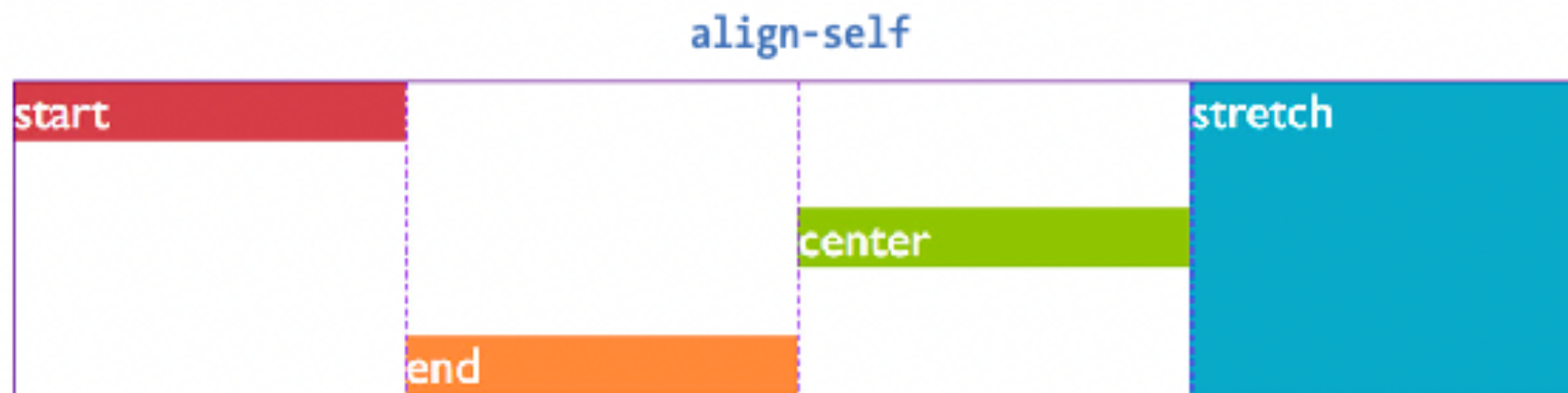
Default: auto (looks at the value for align-items)

Applies to: grid items

Inherits: no

- **Note:** The [self-start](#) and [self-end](#) values look at the writing direction of the content of the item. The [start](#) and [end](#) values consider the writing direction of the grid container. The [left](#) and [right](#) are absolute and would not change with the writing system.

Align individual items



Aligning All the Items

justify-items

Values:	start end center left right self-start self-end stretch normal
Default:	normal (stretch for non-replaced elements; start for replaced elements)
Applies to:	grid containers
Inherits:	no

align-items

Values:	start end center left right self-start self-end stretch normal
Default:	normal (stretch for non-replaced elements; start for replaced elements)
Applies to:	grid containers
Inherits:	no

Note: The stretch keyword works only when the track width or height is set to auto.

Aligning tracks in the grid container

justify-content

Values: [start](#) | end | center | left | right | stretch | space-around | space-between | space-evenly

Default: start

Applies to: grid containers

Inherits: no

align-content

Values: [start](#) | end | center | left | right | stretch | space-around | space-between | space-evenly

Default: start

Applies to: grid containers

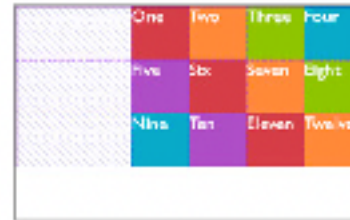
Inherits: no

Track Alignment

justify-content:
start



end



center



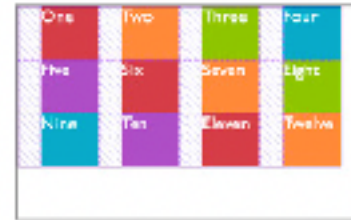
space-around



space-between



space-evenly



align-content:
start



end



center



space-around



space-between



space-evenly



Grid Property Review

Grid container properties

`display: grid | inline-grid`
`grid`
 `grid-template`
 `grid-template-rows`
 `grid-template-columns`
 `grid-template-areas`
 `grid-auto-rows`
 `grid-auto-columns`
 `grid-auto-flow`
`grid-gap`
 `grid-row-gap`
 `grid-column-gap`
`justify-items`
`align-items`
`justify-content`
`align-content`

Grid item properties

`grid-column`
 `grid-column-start`
 `grid-column-end`
`grid-row`
 `grid-row-start`
 `grid-row-end`
`grid-area`
`justify-self`
`align-self`
`order` (not part of Grid Module)
`z-index` (not part of Grid Module)